

Практическая работа 1 по курсу
"Базы данных"
Вариант 7
Модель "Заказ билетов"

Мурадян Артём Андреевич
БПМ232

8 марта 2026 г.

Содержание

1	Описание таблиц	3
1.1	Концептуальная описание	3
1.2	Описание таблиц	3
2	Запросы на языке реляционной алгебры	4
3	Группы пользователей и ожидаемые запросы	8
3.1	Менеджер по продажам	8
3.2	Сотрудник службы безопасности	8
3.3	Аналитик	8
3.4	Пассажир	8
4	Нормализация базы данных	8
5	Скрипт создания таблиц на SQL	8
6	Заполнение базы данных	10
7	Реализация отчетов	10

1 Описание таблиц

1.1 Концептуальная описание

Наша БД состоит из следующих сущностей:

- **Пассажир (Passenger)** — информация о людях, которые летят.
- **Рейс (Flight)** — информация о конкретном рейсе (номер, маршрут, время, самолет).
- **Тип самолета (AircraftType)** — справочник моделей самолетов.
- **Тип рейса (FlightType)** — справочник (регулярный, чартер, специальный).
- **Билет (Ticket)** — связь между пассажиром и рейсом, содержит место и статус.

1.2 Описание таблиц

Таблица 1: Описание таблиц базы данных

Имя атрибута	Тип атрибута	Ограничение	Описание атрибута
Таблица Passenger (Пассажиры)			
passenger_id	INTEGER	SERIAL PRIMARY KEY	Уникальный идентификатор пассажира
last_name	VARCHAR(100)	NOT NULL	Фамилия
first_name	VARCHAR(100)	NOT NULL	Имя
patronymic	VARCHAR(100)		Отчество
passport_number	VARCHAR(20)	UNIQUE NOT NULL	Номер паспорта
birth_date	DATE	NOT NULL	Дата рождения
Таблица AircraftType (Типы самолетов)			
aircraft_type_id	INTEGER	SERIAL PRIMARY KEY	Идентификатор типа
model	VARCHAR(50)	UNIQUE NOT NULL	Модель самолета
manufacturer	VARCHAR(50)	NOT NULL	Производитель
capacity	INTEGER	NOT NULL CHECK (capacity > 0)	Количество мест
Таблица FlightType (Типы рейсов)			
flight_type_id	INTEGER	SERIAL PRIMARY KEY	Идентификатор типа
type_name	VARCHAR(50)	UNIQUE NOT NULL	Название типа
Таблица Flight (Рейсы)			
flight_id	INTEGER	SERIAL PRIMARY KEY	Уникальный идентификатор рейса

Имя атрибута	Тип атрибута	Ограничение	Описание атрибута
flight_number	VARCHAR(10)	NOT NULL	Номер рейса
departure_city	VARCHAR(100)	NOT NULL	Город отправления
arrival_city	VARCHAR(100)	NOT NULL	Город прибытия
departure_time	TIMESTAMP	NOT NULL	Дата и время вылета
arrival_time	TIMESTAMP	NOT NULL	Дата и время прилета
aircraft_type_id	INTEGER	REFERENCES AircraftType(aircraft_type_id)	Тип самолета
flight_type_id	INTEGER	REFERENCES FlightType(flight_type_id)	Тип рейса
Таблица Ticket (Билеты)			
ticket_id	INTEGER	SERIAL PRIMARY KEY	Уникальный номер билета
passenger_id	INTEGER	REFERENCES Passenger(passenger_id) ON DELETE CASCADE	Пассажир
flight_id	INTEGER	REFERENCES Flight(flight_id) ON DELETE CASCADE	Рейс
seat_number	VARCHAR(5)		Номер места
is_return_ticket	BOOLEAN	DEFAULT FALSE	Признак обратного билета
purchase_date	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Дата покупки
ticket_status	VARCHAR(20)	DEFAULT 'Куплен'	Статус

2 Запросы на языке реляционной алгебры

Обозначим отношения:

- *Passenger* — пассажиры
- *Flight* — рейсы
- *Ticket* — билеты
- *AircraftType* — типы самолетов
- *FlightType* — типы рейсов

Запрос 1: Список пассажиров заданного рейса

Пусть задан рейс с номером 'SU100':

$$R_1 = \text{Flight} \bowtie \text{Ticket} \bowtie \text{Passenger}$$

$$\text{Result} = \pi_{\text{LastName, FirstName, Patronymic, PassportNumber}}(\sigma_{\text{FlightNumber}='SU100'}(R_1))$$



Запрос 2: Пассажиры с обратным билетом до заданного города

Пусть заданный город — 'Москва':

$$R_1 = Flight \bowtie Ticket \bowtie Passenger$$
$$Result = \pi_{LastName, FirstName, Patronymic}(\sigma_{ArrivalCity='Moscow' \wedge IsReturnTicket=TRUE}(R_1))$$

Запрос 3: Рейсы пассажира по фамилии и дате

Пусть фамилия — 'Иванов', дата — '2024-03-15':

$$R_1 = Passenger \bowtie Ticket \bowtie Flight$$
$$Result = \pi_{FlightNumber, DepartureCity, ArrivalCity, DepartureTime}$$
$$(\sigma_{LastName='Ivanov' \wedge DATE(DepartureTime)='2024-03-15'}(R_1))$$

Запрос 4: Однофамильцы на одном рейсе

$$R_1 = Passenger \bowtie Ticket$$
$$R_2 = R_1 \times R_1$$
$$R_3 = \sigma_{R_1.LastName=R_2.LastName \wedge R_1.FlightID=R_2.FlightID \wedge R_1.PassengerID < R_2.PassengerID}(R_2)$$
$$Result = \pi_{R_1.LastName, R_1.FirstName, R_2.FirstName, R_1.FlightNumber, R_1.DepartureTime}(R_3)$$

Запрос 5: Список свободных мест

Для этого запроса необходимо ввести вспомогательное отношение *Seats*, содержащее все возможные номера мест:

$$Seats = \pi_{SeatNumber}(\sigma_{SeatNumber \text{ IS NOT NULL}}(Ticket))$$

Тогда все возможные комбинации рейсов и мест:

$$AllFlightSeats = Flight \times Seats$$

Занятые места (билеты с указанными местами):

$$OccupiedSeats = \pi_{FlightID, SeatNumber}(Ticket)$$

Свободные места получаются вычитанием занятых из всех возможных:

$$FreeSeats = AllFlightSeats \setminus OccupiedSeats$$

Итоговый результат с проекцией на нужные атрибуты:

$$Result = \pi_{DepartureDate, FlightNumber, Seat, DepartureCity, ArrivalCity}(FreeSeats \bowtie Flight)$$

где $DepartureDate = DATE(DepartureTime)$.

Запрос 6: Пассажиры до заданного города в интервале дат

Пусть заданный город — 'Сочи', интервал — с '2024-03-01' по '2024-03-31':

$$\begin{aligned} R_1 &= Flight \bowtie Ticket \bowtie Passenger \\ Result &= \pi_{LastName, FirstName, Patronymic, FlightNumber, DepartureTime} \\ &(\sigma_{ArrivalCity='Sochi' \wedge DepartureTime \geq '2024-03-01' \wedge DepartureTime \leq '2024-03-31'}(R_1)) \end{aligned}$$

Запрос 7: Типы самолетов по маршруту пассажира

Пусть задан номер билета $ticket_id = 1$:

Сначала найдем рейс, на который куплен данный билет:

$$R_1 = \sigma_{TicketID=1}(Ticket) \bowtie Flight$$

Из этого рейса получим маршрут (город отправления и прибытия):

$$Route = \pi_{DepartureCity, ArrivalCity}(R_1)$$

Теперь найдем все рейсы, выполняющиеся по этому маршруту:

$$R_2 = Flight \bowtie Route$$

И для этих рейсов получим типы самолетов:

$$R_3 = R_2 \bowtie AircraftType$$

$$Result = \pi_{Model, Manufacturer}(R_3)$$

Запрос 8: Три самых востребованных направления за месяц

Пусть задан месяц — март 2024 года (год = 2024, месяц = 3):

Сначала отфильтруем рейсы за указанный месяц:

$$\begin{aligned} MonthlyFlights &= \sigma_{EXTRACT(YEARFROM DepartureTime)=2024 \wedge EXTRACT(MONTHFROM DepartureTime)=3} \\ &(Flight) \end{aligned}$$

Подсчитаем количество проданных билетов по каждому направлению:

$$R_1 = MonthlyFlights \bowtie Ticket$$

$$DirectionCounts = \mathcal{G}_{DepartureCity, ArrivalCity; COUNT(DISTINCT TicketID) \rightarrow TicketCount}(R_1)$$

Сортируем по убыванию количества и берем первые 3:

$$SortedDirections = \tau_{TicketCountDESC}(DirectionCounts)$$

$$Result = \delta_3(SortedDirections)$$

где $\mathcal{G}$ — оператор группировки с агрегатными функциями, τ — оператор сортировки, δ_3 — оператор ограничения количества записей (TOP 3).

3 Группы пользователей и ожидаемые запросы

3.1 Менеджер по продажам

- Сколько билетов продано на рейс SU100 за последнюю неделю?
- Какие пассажиры сегодня вылетают в Париж?

3.2 Сотрудник службы безопасности

- Есть ли в списке пассажиров на рейс SU200 однофамильцы с подозреваемыми (заданный список фамилий)?
- Кто из пассажиров с фамилией Иванов летал за последний месяц?

3.3 Аналитик

- Какие 3 направления были самыми популярными в прошлом месяце?
- Какой тип самолета чаще всего используется на рейсах в Новосибирск?

3.4 Пассажир

- Есть ли свободные места на рейс SU300 завтра?
- Какие рейсы летят из Москвы в Сочи в ближайшую пятницу?

4 Нормализация базы данных

Спроектированная база данных соответствует **Третьей нормальной форме (3НФ)**.
Обоснование:

1. **1НФ:** Все атрибуты атомарны. В таблице Passenger ФИО разбито на три отдельных поля. Даты и времена хранятся в стандартных типах данных SQL, что обеспечивает их неделимость в контексте данного отношения.
2. **2НФ:** Таблицы находятся в 1НФ, и каждый неключевой атрибут функционально полно зависит от первичного ключа. В таблице Ticket неключевые поля (seat_number, purchase_date) зависят от всей связки (рейс + пассажир), а не от части ключа.
3. **3НФ:** Таблицы находятся в 2НФ и не содержат транзитивных зависимостей. Например, в таблице Flight информация о модели самолета вынесена в отдельную таблицу AircraftType и связана через внешний ключ. Таким образом, departure_city зависит только от flight_id и не зависит от других неключевых атрибутов.

5 Скрипт создания таблиц на SQL

```
-- Создаем таблицу типов самолетов
CREATE TABLE AircraftType (
    aircraft_type_id SERIAL PRIMARY KEY,
    model VARCHAR(50) UNIQUE NOT NULL,
    manufacturer VARCHAR(50) NOT NULL,
```



```

        capacity INTEGER NOT NULL CHECK (capacity > 0)
    );

-- Создаем таблицу типов рейсов (справочник)
CREATE TABLE FlightType (
    flight_type_id SERIAL PRIMARY KEY,
    type_name VARCHAR(50) UNIQUE NOT NULL
);

-- Создаем таблицу пассажиров
CREATE TABLE Passenger (
    passenger_id SERIAL PRIMARY KEY,
    last_name VARCHAR(100) NOT NULL,
    first_name VARCHAR(100) NOT NULL,
    patronymic VARCHAR(100),
    passport_number VARCHAR(20) UNIQUE NOT NULL,
    birth_date DATE NOT NULL
);

-- Создаем таблицу рейсов
CREATE TABLE Flight (
    flight_id SERIAL PRIMARY KEY,
    flight_number VARCHAR(10) NOT NULL,
    departure_city VARCHAR(100) NOT NULL,
    arrival_city VARCHAR(100) NOT NULL,
    departure_time TIMESTAMP NOT NULL,
    arrival_time TIMESTAMP NOT NULL,
    aircraft_type_id INTEGER REFERENCES AircraftType(aircraft_type_id),
    flight_type_id INTEGER REFERENCES FlightType(flight_type_id)
);

-- Создаем таблицу билетов (связь пассажиров и рейсов)
CREATE TABLE Ticket (
    ticket_id SERIAL PRIMARY KEY,
    passenger_id INTEGER NOT NULL REFERENCES Passenger(passenger_id) ON DELETE CASCADE,
    flight_id INTEGER NOT NULL REFERENCES Flight(flight_id) ON DELETE CASCADE,
    seat_number VARCHAR(5),
    is_return_ticket BOOLEAN DEFAULT FALSE,
    purchase_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    ticket_status VARCHAR(20) DEFAULT 'Куплен',
    CONSTRAINT unique_flight_passenger UNIQUE (flight_id, passenger_id)
);

-- Индексы для ускорения поиска
CREATE INDEX idx_ticket_passenger ON Ticket(passenger_id);
CREATE INDEX idx_ticket_flight ON Ticket(flight_id);
CREATE INDEX idx_flight_datetime ON Flight(departure_time);
CREATE INDEX idx_passenger_name ON Passenger(last_name, first_name);

-- Инициализация всех таблиц

```

```
-- DROP TABLE AircraftType, FlightType, Passenger, Flight, Ticket;
-- DROP TABLE IF EXISTS AircraftType, FlightType, Passenger, Flight, Ticket CASCADE;
```

6 Заполнение базы данных

```
-- Заполняем справочники
```

```
INSERT INTO FlightType (type_name) VALUES
    ('Регулярный'),
    ('Чартер'),
    ('Специальный');
```

```
INSERT INTO AircraftType (model, manufacturer, capacity) VALUES
    ('Boeing 737-800', 'Boeing', 189),
    ('Airbus A320', 'Airbus', 180),
    ('Sukhoi Superjet 100', 'Sukhoi', 98);
```

```
-- Добавляем пассажиров
```

```
INSERT INTO Passenger (last_name, first_name, patronymic, passport_number, birth_date)
    ('Иванов', 'Александр', 'Сергеевич', '1234 567898', '1990-05-15'),
    ('Иванов', 'Иван', 'Иванович', '1234 567890', '1990-05-15'),
    ('Петров', 'Петр', 'Петрович', '2345 678901', '1985-10-20'),
    ('Сидоров', 'Сидр', 'Сидорович', '3456 789012', '1978-03-10'),
    ('Иванова', 'Мария', 'Ивановна', '4567 890123', '1995-07-25'),
    ('Петров', 'Алексей', 'Иванович', '5678 901234', '1988-12-05'),
    ('Иванов', 'Александр', 'Сергеевич', '1234 567899', '1990-05-15');
```

```
-- Добавляем рейсы
```

```
INSERT INTO Flight (flight_number, departure_city, arrival_city, departure_time, arrival_time)
    ('SU100', 'Москва', 'Санкт-Петербург', '2024-03-15 10:00:00', '2024-03-15 11:30:00'),
    ('SU101', 'Санкт-Петербург', 'Москва', '2024-03-18 18:00:00', '2024-03-18 19:30:00'),
    ('SU200', 'Москва', 'Сочи', '2024-03-16 14:00:00', '2024-03-16 16:30:00', 2, 1),
    ('SU201', 'Сочи', 'Москва', '2024-03-20 09:00:00', '2024-03-20 11:30:00', 2, 1),
    ('SH750', 'Москва', 'Анталья', '2024-03-17 23:00:00', '2024-03-18 04:00:00', 3, 2);
```

```
-- Продаем билеты
```

```
INSERT INTO Ticket (passenger_id, flight_id, seat_number, is_return_ticket, purchase_date)
    (6, 1, '1A', FALSE, '2024-02-02 11:20:00'),
    (1, 1, '12A', FALSE, '2024-02-01 10:00:00'),
    (2, 1, '12B', FALSE, '2024-02-01 10:05:00'),
    (1, 2, '10A', TRUE, '2024-02-01 10:00:00'), -- Обратный билет для Иванова
    (3, 3, '5F', FALSE, '2024-02-15 14:30:00'),
    (4, 3, '5E', FALSE, '2024-02-15 14:35:00'),
    (5, 3, '6A', FALSE, '2024-02-16 09:00:00'),
    (2, 5, '1A', FALSE, '2024-03-01 11:20:00');
```

7 Реализация отчетов

```
-- 1. Получить список всех пассажиров заданного рейса (SU100)
```

```
SELECT p.last_name, p.first_name, p.patronymic, p.passport_number
```

```

FROM Passenger p
JOIN Ticket t ON p.passenger_id = t.passenger_id
JOIN Flight f ON t.flight_id = f.flight_id
WHERE f.flight_number = 'SU100';

-- 2. Пассажиры, вылетающие до заданного города и имеющие
-- обратный билет (город = 'Москва')
SELECT DISTINCT p.last_name, p.first_name, p.patronymic
FROM Passenger p
JOIN Ticket t ON p.passenger_id = t.passenger_id
JOIN Flight f ON t.flight_id = f.flight_id
WHERE f.arrival_city = 'Москва' AND t.is_return_ticket = TRUE;

-- 3. Рейсы пассажира с заданной фамилией и датой вылета ('Иванов', '2024-03-15')
SELECT f.flight_number, f.departure_city, f.arrival_city, f.departure_time
FROM Flight f
JOIN Ticket t ON f.flight_id = t.flight_id
JOIN Passenger p ON t.passenger_id = p.passenger_id
WHERE p.last_name = 'Иванов' AND DATE(f.departure_time) = '2024-03-15';

-- 4. Однофамильцы на одном рейсе
SELECT p1.last_name, p1.first_name, p2.first_name AS namesake_firstname,
       f.flight_number, f.departure_time
FROM Passenger p1
JOIN Ticket t1 ON p1.passenger_id = t1.passenger_id
JOIN Passenger p2 ON p1.last_name = p2.last_name
AND p1.passenger_id < p2.passenger_id
JOIN Ticket t2 ON p2.passenger_id = t2.passenger_id AND t1.flight_id = t2.flight_id
JOIN Flight f ON t1.flight_id = f.flight_id;

-- 5. Список свободных мест (Дата, № рейса, № места, Город отправления, Город прибытия)
-- Генерируем все возможные места для рейса и вычитаем занятые
WITH seat_numbers AS (
    SELECT DISTINCT seat_number AS seat
    FROM Ticket
    WHERE seat_number IS NOT NULL
),
flight_seats AS (
    SELECT f.flight_id, f.flight_number, f.departure_city, f.arrival_city,
           f.departure_time::DATE AS flight_date, s.seat
    FROM Flight f
    CROSS JOIN seat_numbers s
)
SELECT fs.flight_date AS "Дата", fs.flight_number AS "№ рейса",
       fs.seat AS "№ места", fs.departure_city AS "Город отправления",
       fs.arrival_city AS "Город прибытия"
FROM flight_seats fs
LEFT JOIN Ticket t ON fs.flight_id = t.flight_id AND fs.seat = t.seat_number
WHERE t.ticket_id IS NULL;

```

```

-- 6. Пассажиры, вылетающие до заданного города в заданном интервале дат
-- (город = 'Сочи', интервал '2024-03-01' - '2024-03-31')
SELECT p.last_name, p.first_name, p.patronymic, f.flight_number, f.departure_time
FROM Passenger p
JOIN Ticket t ON p.passenger_id = t.passenger_id
JOIN Flight f ON t.flight_id = f.flight_id
WHERE f.arrival_city = 'Сочи'
      AND f.departure_time BETWEEN '2024-03-01' AND '2024-03-31';

-- 7. Типы самолетов, летающих по тому же маршруту,
-- что и самолет пассажира с заданным номером билета (ticket_id = 1)
SELECT DISTINCT at2.model, at2.manufacturer
FROM Ticket t1
JOIN Flight f1 ON t1.flight_id = f1.flight_id
JOIN AircraftType at1 ON f1.aircraft_type_id = at1.aircraft_type_id
JOIN Flight f2 ON f1.departure_city = f2.departure_city
AND f1.arrival_city = f2.arrival_city
JOIN AircraftType at2 ON f2.aircraft_type_id = at2.aircraft_type_id
WHERE t1.ticket_id = 1;

-- 8. Три самых востребованных направления за указанный месяц
-- (март 2024)
SELECT f.departure_city AS "Город отправления",
       f.arrival_city AS "Город прибытия",
       COUNT(DISTINCT t.ticket_id) AS "Количество проданных билетов"
FROM Flight f
JOIN Ticket t ON f.flight_id = t.flight_id
WHERE EXTRACT(YEAR FROM f.departure_time) = 2024
      AND EXTRACT(MONTH FROM f.departure_time) = 3
GROUP BY f.departure_city, f.arrival_city
ORDER BY COUNT(DISTINCT t.ticket_id) DESC
LIMIT 3;

```